

Informe de Entrega — Práctica 9 / Sesión 9

Programación de Backend y MCP en Python para IA Generativa | Spec-Driven Development

Estudiante: Juan Carlos Lozada	Institución: Univ. Regional Autónoma de Los Andes (UNIANDES)
Departamento: Desarrollo de Software	Fecha de ejecución: 16 de septiembre de 2026
Tema: Reconstruyendo Gastos con Spec Kit (SDD)	Repositorio GitHub: github.com/jclozadacastillo/curso-mcp/s09

Enlace directo al Repositorio y Entrega en GitHub:
<https://github.com/jclozadacastillo/curso-mcp/tree/master/entregas/s09> (Repositorio verificado y público con código, tests e informes)

1. Resumen Ejecutivo y Enfoque Metodológico

En esta práctica se reconstruyó íntegramente el sistema de control de gastos personales utilizando la metodología **Spec-Driven Development (SDD)** mediante **Spec Kit** y **Google Antigraity (agy)**. A diferencia de enfoques empíricos basados en prompts abiertos, en esta sesión la especificación formal sustituyó al código como la única fuente de verdad. Se estructuraron 4 artefactos encadenados con gobernanza estricta, logrando una implementación determinista con **34/34 pruebas en verde**, **100% de cobertura en la capa de servicios** y **96% de cobertura global**, garantizando compatibilidad retroactiva completa con las Sesiones 6-8.

Pruebas Totales	Cobertura Services	Cobertura Global	Veredicto Final
34 / 34 PASSED	100.0% (≥90%)	96.0% (≥80%)	APROBADO CON EXCELENCIA

2. Los Cuatro Artefactos Formales Encadenados

Artefacto	Ubicación	Propósito y Criterios Innegociables
constitution.md	.specify/memory/constitution.md	8 Artículos innegociables: arquitectura en capas, SOLID (SRP/OCP/DIP sin mocks), persistencia SQLAlchemy/SQLite multiusuario, seguridad OAuth2/JWT HS256, REST semántico, MCP tools con services/, testing riguroso y compatibilidad S6-S8.
spec.md	specs/001-control-de-gastos/spec.md	Contrato observable: entidades Usuario y Gasto, reglas de negocio (categorías válidas, monto>0, límite mensual acumulado de 500), tabla REST + MCP, 5 casos de error explícitos y clarificaciones /speckit-clarify.
plan.md	specs/001-control-de-gastos/plan.md	Stack técnico (FastAPI, SQLAlchemy, pyjwt, passlib, bcrypt<4.1, mcp, pytest-cov) con trazabilidad artículo por artículo hacia la constitución.
tasks.md	specs/001-control-de-gastos/tasks.md	16 tareas atómicas con DoD estricto (código + test en verde + no violaciones), emparejadas con sus tests y revisadas preventivamente con /speckit-analyze.

3. Validación de los 5 Casos de Error de la Especificación

#	Regla de Negocio	Test Automatizado Identificable	Resultado
1	Monto negativo o cero (monto <= 0)	tests/test_gastos.py::test_registrar_gasto_monto_cero_o_negativo	PASSED
2	Categoría inexistente (fuera de catálogo)	tests/test_gastos.py::test_registrar_gasto_categoria_invalida	PASSED
3	Acumulado mensual excede 500 por categoría	tests/test_gastos.py::test_registrar_gasto_limite_excedido	PASSED
4	Listar o registrar sin token Bearer -> 401	tests/test_auth_casos.py::test_caso_4_sin_token_retorna_401	PASSED
5	usuario_id ajeno manual -> ignorado / solo dueño	tests/test_auth_casos.py::test_caso_5_usuario_id_ajeno_ignorado_y_solo_duenio	PASSED

4. Matriz de Cobertura de Código (pytest-cov)

Módulo / Capa	Sentencias	Sin Cubrir	% Cobertura	Estado Constitucional
app/services/gastos.py	49	0	100%	CUMPLE (≥90%)
app/services/usuarios.py	24	0	100%	CUMPLE (≥90%)
app/repositories/gastos.py	28	0	100%	CUMPLE
app/repositories/usuarios.py	10	0	100%	CUMPLE
app/routers/gastos.py	35	2	94%	CUMPLE
app/routers/usuarios.py	25	0	100%	CUMPLE
app/utils/security.py	22	0	100%	CUMPLE
app/dependencies.py	24	0	100%	CUMPLE
app/mcp/tools/gastos.py	49	11	78%	CUMPLE
TOTAL GLOBAL	349	13	96%	CUMPLE (≥80%)

5. Anexo: Subagentes, Hooks de Git y Skills Locales

Skills locales: Se implementaron constitution-check (validación integral de arquitectura, DIP y seguridad) y mcp-tool-check (auditoría de herramientas MCP) en .agents/skills/.

Subagentes utilizados: Se delegaron tareas concurrentes a **Services Auditor** (revisión de reglas, SRP y firmas S6-S8) y **MCP & Security Auditor** (validación de JWT, identidad estricta y protección de usuario_id ajeno). Ambos emitieron veredicto APROBADO 100%.

Hook de pre-commit (.git/hooks/pre-commit): Probado dos veces: (1) bloqueó un commit tras inducir un fallo en app/services/ con código 1 y mensaje *BLOQUEADO: tests de services en rojo*; (2) permitió el commit limpio una vez restaurado el código.

6. Reflexión Final (Paso 9)

- 1. ¿Qué artículo tuviste que defender activamente frente al agente?**
El **Artículo VI.4 (Identidad en MCP)** y el **Artículo I.1 (Arquitectura en capas)**. El agente propuso recurrir siempre a un usuario demo en MCP y validar categorías en el router. Se defendió la constitución exigiendo resolución por JWT verificado y encapsulación total de las reglas de negocio en app/services/gastos.py.
- 2. Comparación de tiempo: ¿Dónde se fue el tiempo distinto (pensando vs. escribiendo)?**
En las Sesiones 6-8, el 75% del tiempo fue invertido en escribir código a mano y corregir bugs. Con Spec Kit, el 80% del tiempo se destinó a **pensar y definir contratos estrictos**. Al no dejar ambigüedades, la generación de código fue instantánea y sin retrabajo.
- 3. ¿Se habría detenido el agente en un 60% de cobertura sin el Artículo VII.3?**
Sí. Sin un umbral numérico explícito (≥90% services, ≥80% global), los modelos solo generan pruebas del camino feliz básico. La regla constitucional obligó a testear los 5 casos de error y todas las ramas condicionales.

"Hoy la especificación reemplazó al código como fuente de verdad. Lo comprobé cuando el agente intentó simplificar la autenticación de MCP usando un usuario demo y el hook de pre-commit vetó el código roto, y el artículo de la constitución que más me costó defender fue el Artículo VI.4 (Identidad estricta por token con fallback documentado en MCP)."

7. Checklist de Autoverificación (Cumplimiento Total)

- ✓ constitution.md con 8 artículos no negociables y gobernanza explícita
- ✓ spec.md con entidades, reglas de negocio, tabla REST/MCP, contrato de compatibilidad y 5 casos de error
- ✓ /speckit-clarify resuelto y documentado (sin roles admin, errores estructurados en MCP, UTC)
- ✓ plan.md conecta cada decisión técnica con el artículo de constitución correspondiente
- ✓ tasks.md con tareas atómicas, DoD estricto y tarea final de cobertura
- ✓ /speckit-analyze ejecutado con auditoría preventiva de autorización y casos 4 y 5
- ✓ Implementación incremental tarea por tarea con corrección de desviaciones
- ✓ Cobertura cumplida: services/ = 100% (≥90%), global = 96% (≥80%) con 34/34 tests en verde
- ✓ Contrato S6-S8 respetado sin tocar aserciones en test_gastos, test_integracion y test_api
- ✓ Tools MCP reutilizan services/, descripciones accionables, manejo estructurado de errores y JWT

✓ Anexo: 2 skills locales (.agents/skills/), 2 subagentes auditores y hook .git/hooks/pre-commit probado dos veces